



كلية العلوم
السملاية - مراكش
FACULTÉ DES SCIENCES
SEMLALIA - MARRAKECH

UNIVERSITÉ CADI AYYAD

FACULTÉ DES SCIENCES SEMLALIA (FSSM)

DEPARTEMENT INFORMATIQUE

Rapport de Projet Réalisé dans le Cadre des Modules
Programmation Web JEE Spr et Progr Web & Mobile Full S

Plateforme de Gestion des Rattrapages

Réservation de Salles et Planification Pédagogique

Réalisé par :

Gouiga YASSINE
Karimane MOUHCINE
Id Omar OUSSAMA

Encadré par :

Pr. AATILA
Pr. JARIR

Année Universitaire : 2025/2026

Table des matières

1	Introduction et Contexte du Projet	2
2	Choix Technologiques et Architecture	2
2.1	Backend : Spring Boot 3 et Java 17	2
2.2	Frontend : React.js et Tailwind CSS	3
3	Modélisation de la Base de Données	3
4	Fonctionnalités : Espace Administrateur	4
4.1	Tableau de Bord et Métriques	4
4.2	Gestion du Corps Enseignant et des Salles	4
4.3	Supervision et Modération	4
5	Fonctionnalités : Espace Enseignant	5
5.1	Recherche Algorithmique et Réservation	5
5.2	Gestion Personnelle des Séances	5
6	Défis Techniques et Solutions Apportées	6
7	Conclusion et Perspectives	6

1 Introduction et Contexte du Projet

L'organisation des séances de rattrapage au sein de la Faculté des Sciences Semlalia de l'Université Cadi Ayyad est une tâche administrative et pédagogique complexe. Historiquement, la gestion de l'occupation des salles repose souvent sur des processus manuels ou semi-automatisés. Ces méthodes traditionnelles peuvent engendrer des conflits d'horaires, des doubles réservations, et ralentir considérablement la communication entre l'administration et le corps enseignant.

Pour pallier ces difficultés, ce projet a été initié avec pour objectif le développement d'une plateforme web robuste, centralisée et ergonomique, baptisée "Gestion Rattrapages".

Les objectifs principaux de cette solution sont :

- **Digitalisation et Centralisation** : Remplacer les registres papiers ou les tableaux par une base de données unique et fiable.
- **Autonomie des Enseignants** : Permettre aux professeurs de vérifier la disponibilité des salles et de réserver des créneaux en temps réel sans intervention de l'administration.
- **Supervision Administrative** : Offrir à l'administration un outil de contrôle pour gérer les ressources (activation/désactivation de salles) et superviser l'ensemble des activités pédagogiques.

2 Choix Technologiques et Architecture

Afin de garantir des performances optimales, une maintenance simplifiée et une évolutivité à long terme, nous avons opté pour une architecture distribuée (Architecture Multi-Tiers). Cette approche sépare strictement la logique métier (Backend) de l'interface utilisateur (Frontend).

2.1 Backend : Spring Boot 3 et Java 17

L'API RESTful a été développée en Java à l'aide du framework Spring Boot 3.

- **Architecture interne** : Le code est structuré selon le modèle MVC (Modèle - Vue/Contrôleur - Service - Repository), garantissant un découplage fort entre l'accès aux données et la logique métier.
- **Accès aux données** : Spring Data JPA (basé sur Hibernate) a été utilisé comme ORM (Object-Relational Mapping). Cela permet de manipuler la base de données via des objets Java, évitant ainsi la rédaction manuelle de requêtes SQL complexes, tout en prévenant les injections SQL.
- **Sécurité (Spring Security & JWT)** : L'authentification est dite *Stateless*. Lorsqu'un utilisateur se connecte, le serveur génère un jeton JWT (JSON Web Token) signé cryptographiquement. Ce jeton est ensuite exigé pour chaque requête HTTP, assurant que seules les personnes autorisées peuvent accéder aux ressources. Les rôles (Admin/Enseignant) sont vérifiés au niveau des contrôleurs grâce à l'annotation `@PreAuthorize`.

2.2 Frontend : React.js et Tailwind CSS

Côté client, nous avons développé une Single Page Application (SPA) offrant une expérience utilisateur fluide, sans rechargement de page.

- **React.js et Vite** : Utilisé pour concevoir des composants réutilisables. Le projet a été initialisé avec Vite.js pour bénéficier d'un environnement de développement ultra-rapide.
- **Gestion de l'état (Context API)** : Un `AuthContext` a été mis en place pour mémoriser globalement l'état de l'utilisateur (connecté, rôle, informations) et protéger les routes sensibles via un composant `ProtectedRoute`.
- **Tailwind CSS** : L'interface a été stylisée selon les principes du "Utility-First". Cela nous a permis d'implémenter un design "Premium" (effets de flou d'arrière-plan ou *Glassmorphism*, ombres douces) et d'assurer une compatibilité 100% Mobile-Responsive.

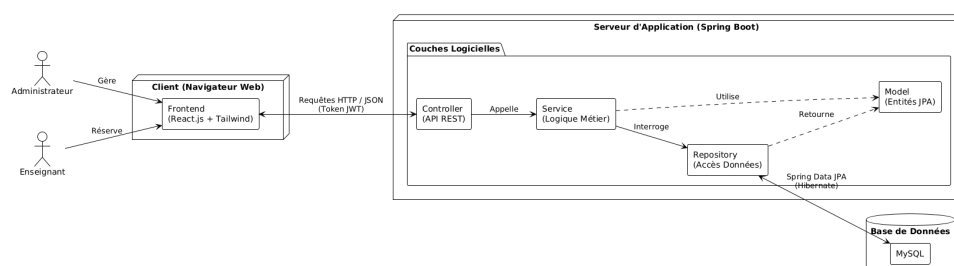


FIGURE 1 – Architecture technique et flux des requêtes HTTP

3 Modélisation de la Base de Données

L'application repose sur un SGBD MySQL. Le modèle de données a été normalisé pour garantir l'intégrité référentielle et éviter toute redondance. Les relations clés sont les suivantes :

- **User (Utilisateur)** : Héberge les identifiants de connexion (email unique, mot de passe hashé par BCrypt) et le nom complet.
- **Role (Rôle)** : Une relation *Many-to-Many* lie les utilisateurs aux rôles (`ROLE_ADMIN`, `ROLE_ENSEIGNANT`), gérée par une table de jointure `user_roles`.
- **Salle** : Entité physique définie par un identifiant, un nom, une capacité d'accueil et un statut (actif/inactif).
- **Reservation** : L'entité transactionnelle centrale. Elle associe une Salle à un Utilisateur via des relations *Many-to-One*. Elle stocke la date (`dateRes`), l'heure de début, l'heure de fin et un motif.

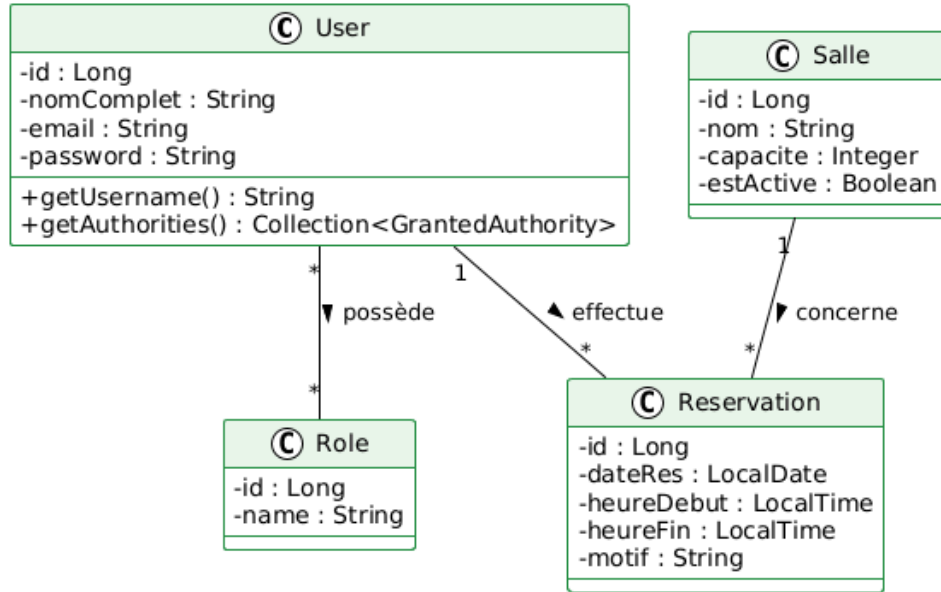


FIGURE 2 – Diagramme de classes (UML) illustrant les relations JPA

4 Fonctionnalités : Espace Administrateur

La plateforme réserve un espace hautement privilégié à l’administration, accessible uniquement aux comptes dotés du rôle `ROLE_ADMIN`.

4.1 Tableau de Bord et Métriques

Dès la connexion, l’administrateur accède à un tableau de bord (Dashboard) affichant des indicateurs clés de performance (KPI) calculés en temps réel : nombre total d’enseignants inscrits, nombre de salles actives, et volume global des réservations.

4.2 Gestion du Corps Enseignant et des Salles

L’administration a le contrôle total sur l’inscription. Un formulaire modal permet de créer de nouveaux comptes professeurs de manière sécurisée. Concernant l’infrastructure, l’administrateur peut lister, ajouter et surtout **désactiver** des salles de classe. Une salle désactivée (par exemple pour travaux de maintenance ou nettoyage) disparaît immédiatement des résultats de recherche côté enseignants, prévenant ainsi toute réservation abusive.

4.3 Supervision et Modération

Une interface de supervision sous forme de tableau interactif permet de suivre l’ensemble des réservations passées et futures de la faculté. En cas de force majeure, l’administrateur possède le privilège d’annuler manuellement n’importe quelle réservation pour libérer le créneau.

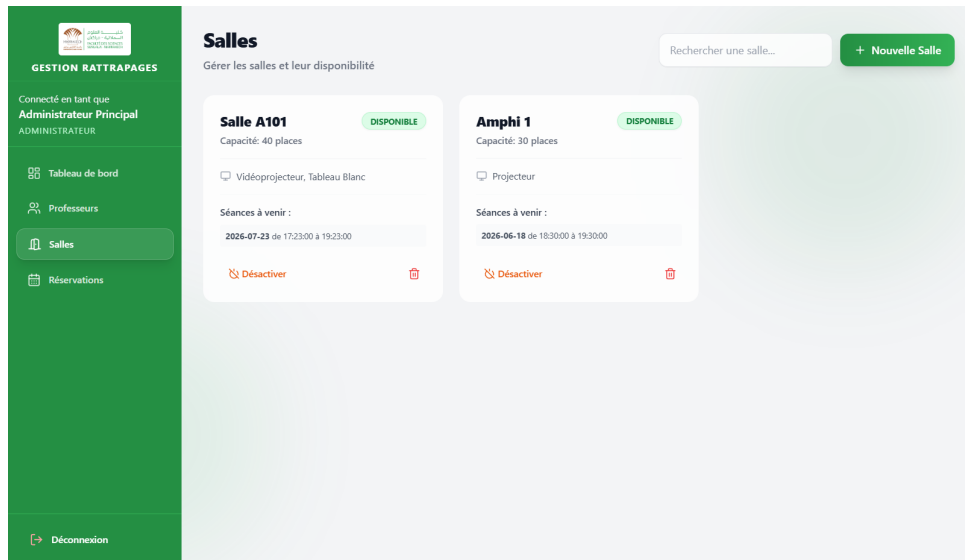


FIGURE 3 – Vue de l'Espace Administrateur (Gestion des Salles)

5 Fonctionnalités : Espace Enseignant

L'espace destiné au corps enseignant a été pensé pour réduire au maximum le nombre de clics nécessaires à la planification d'un cours.

5.1 Recherche Algorithmique et Réservation

Pour réserver, l'enseignant saisit une date, une heure de début et une heure de fin. Lors de la soumission du formulaire, une requête est envoyée au serveur. L'algorithme Backend croise ces horaires avec les réservations existantes et ne retourne au Frontend que les salles strictement ****disponibles****. L'enseignant visualise ces résultats sous forme de "Cartes", choisit la salle adéquate, ajoute un motif (ex : "Rattrapage d'Algorithmique"), et valide.

5.2 Gestion Personnelle des Séances

Dans l'onglet "Mes Réservations", le professeur dispose de son propre historique. Le tableau distingue clairement (par code couleur) les séances "À venir" des séances "Passées". L'enseignant jouit d'une autonomie totale pour annuler ses propres réservations futures s'il décide de reporter sa séance.

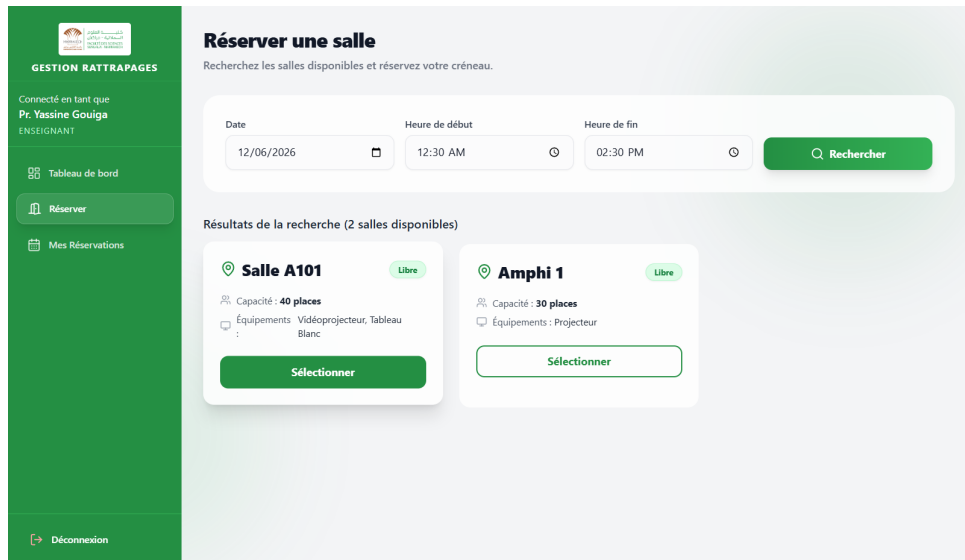


FIGURE 4 – Processus de réservation d’une salle par un enseignant

6 Défis Techniques et Solutions Apportées

La réalisation de cette plateforme a nécessité la résolution de plusieurs problématiques d’ingénierie :

1. **Prévention des Doubles Réservations** : Pour garantir l’unicité de l’occupation d’une salle, nous avons conçu une requête personnalisée (@Query) dans le *ReservationRepository*. Cette requête vérifie s’il existe déjà une réservation pour la même salle, à la même date, dont l’heure de début est strictement inférieure à l’heure de fin souhaitée, et dont l’heure de fin est strictement supérieure à l’heure de début souhaitée.
2. **Problème de Récursivité JSON** : En raison des relations bidirectionnelles JPA (Un *User* possède des *Reservations*, qui possèdent elles-mêmes un *User*), la conversion des objets en JSON causait des boucles infinies (StackOverflow). Nous avons résolu cela via l’utilisation ciblée de l’annotation @JsonIgnore sur certains attributs enfants.
3. **Adaptabilité Mobile (Responsive UI)** : Assurer la lisibilité de tableaux complexes sur des écrans de smartphones a nécessité l’emploi de conteneurs `overflow-x-auto` et la transformation dynamique du menu latéral en tiroir (*Drawer*) pour les petits écrans.

7 Conclusion et Perspectives

Ce projet a consolidé nos acquis en ingénierie logicielle autour d’un cas d’usage concret pour l’Université Cadi Ayyad. La plateforme est aujourd’hui robuste, sécurisée et dotée d’une UI moderne. Les évolutions futures pourraient inclure l’intégration de la librairie JavaMailSender pour des alertes automatiques par e-mail, ou la possibilité de planifier des réservations récurrentes.